

Amorphous Transdisciplinary Engineering: Object Orientation Meets Service Orientation with Emergent Multifidelity Management

Michael Sobolewski

Air Force Research Laboratory, WPAFB, Ohio 45433

Polish Japanese Academy of IT, 02-008 Warsaw, Poland

e-mail: sobol@sorcersoft.org

Abstract. Amorphous transdisciplinary engineering is the application of back-end object-oriented provider services and front-end service-oriented micro- and macroservices. Macroservices define dynamic aggregates of large number of provider services (applications, tools, and utilities) with emergent multifidelity management. On the one hand, provider services are local/remote objects having independent limited functionalities that are dynamically bound to microservices. On the other hand, macroservices are request services in terms of microservices and other macroservices with metafidelities that specify required federations of provider services. Metafidelities are fidelities of fidelities. A metafidelity is used by a fidelity management system to select (morph) for a macroservice a collection of microservices at runtime. A macroservice request is called a mograms – model or program, or both. Service mograms created by the end users at the system of system level exhibit emergent behavior of service federations at runtime. A mogram's service federation with associated fidelity management is continuously morphing to the most effective collections of provider services while skipping unsatisfactory transient federations at runtime.

Keywords. Service orientation, microservices, macroservices, provider services, multifidelities, metafidelities, metacomputing, MOF, MSF, system of systems, mogramming, exertions, SML, SORCER

Introduction

Amorphous transdisciplinary engineering (ATE) is the broad usage of heterogeneous computer software for both standalone and distributed systems to aid in complex and adaptive engineering analysis tasks. It integrates various tools from multiple engineering domains including Finite Element Analysis (FEA), Computational Fluid Dynamics (CFD), Multibody dynamics (MBD), and Multidisciplinary Analysis and Design Optimization (MADO).

MADO is a domain of research that studies the dynamic application of numerical analysis and optimization techniques for the design of engineering systems involving multiple disciplines. The formulation of MADO problems has become increasingly complex as the number of engineering disciplines and design variables included in

typical studies has grown from a few dozen to thousands when applying high-fidelity physics-based modeling early in the design process [1].

Software tools that have been developed to support MADO activities are considered ATE tools. There are several trends that are forcing system architectures to evolve due to complexity of engineering problems being solved presently [1]. Users expect a rich, interactive and dynamic user experience on a wide variety of user agents (clients) and highly modular and dynamic systems. Systems must be highly scalable, highly available and run locally or in the network, or both. Organizations often want to frequently roll out updates, even multiple times a day. Consequently, it's no longer adequate to develop simple, monolithic web applications that serve up HTML to desktop browsers. When the dynamic system changes frequently the static user agent cannot catch-up with the backend changes, so it becomes quickly obsolete in evolving complex adaptive systems. In a dynamic system when its backend is morphing constantly - *amorphous system* - the user agent has to be amorphous accordingly. A service-oriented (SO) amorphous system means also *net-centric* to refer to participating as a part of a continuously evolving complex community of people, devices, information and services interconnected by a communications network to achieve optimal benefit of resources and better synchronization of events and their consequences. An amorphous system means also *scalable* and *autonomic* so your communication network of services can be scaled up and down reliably, from a single computer to a large number of computers as needed at runtime.

Recent most popular distributed architectures are based on a service-oriented architecture (SOA) [2]. It is an architectural pattern in computer software design in which application components provide services to other components via a communication protocol, typically over a network. However, SOA-based systems are still very difficult for most users to access, and that detailed and low-level programming must be carried out by the user through command line and script execution to carefully tailor static interactions on each end to the distributed resources on which they will run, or for the data structure that they will access. Most of these architectures in fact represent Service-Disoriented Architecture (SDA). In SDA the notion of service is ill defined with no clear properties such as instantiation, multifidelity, inheritance, polymorphism, composition, delegation, well-defined service lifecycle, etc.; not mentioning confusing SO terminology that is oriented usually on something else except uniform services, in contrast for example to uniform objects in object-oriented systems. With that in mind in Section 1 more formal definitions are provided for uniform front/back-end services and federated amorphous systems.

The Service-ORiented Computing EnviRonment (SORCER) [2] adheres to a true SO architecture that has been developed and applied to solve multidisciplinary design-optimization problems with multifidelity services allowing for integration multiple data sources and tools as morphing service federation. Its true service-orientation means that all system data and operations are uniform services. The unique architecture is about organizational complexity and system complexity that allows treating local and remote service uniformly at various levels of granularity. When dealing with both complexities, you have a case to distribute, otherwise create a modular monolith with locally executable modules as local services. Later, when complexity of the system becomes too complex to manage you can deploy almost instantly the existing local modules as network services on an as-needed basis and then run selected modules of the original monolith in the network. Such SORCER application is accomplished using service

signatures that require a change of service types only or just selecting service fidelities from local to remote when requesting services.

The remainder of this paper is organized as follows: Section 1 describes a conceptual service-oriented framework of federated amorphous systems; Section 2 describes the reference architecture of the modeling and execution framework; Section 3 describes N³ diagramming and mogramming MADO systems; finally Section 4 concludes with final remarks and comments.

1. Service Orientation and Federated Amorphous Systems

In general, a *computing service* is the work performed in which a service provider (one that serves) exerts acquired abilities to execute a computation. SO architecture (SOA) is an architectural pattern in computer software design in which application components provide services to other components via a communications protocol, typically via remote procedure call (RPC). Application components that provide services to other components are called *provider services*. Well-defined SOA is a *service-to-service* (S2S) application architecture that partitions workloads among uniform provider services (service peers).

Six generations of RPC can be distinguished including a federated method invocation (FMI) [1]. The signature based FMI has been used to develop a federated SO environment [2] based on the conceptual SO framework described in this section.

1.1. Service-oriented Terms and Definitions

Federated SO computing studies *computational federations* that are dynamic collections of collaborating services. Five types of basic services are distinguished: context services, microservices, multifidelity services, macroservices, and provider services. *Context services* provide *data* other services operate on. *Microservices* are elementary services. *Provider services* are realizations of microservices; it means that for each microservice there is at least one corresponding actualization – a *provider service*. *Macroservices* are compound services – compositions of microservices and other macroservices. Macroservices represent abstract service models that are actualized by corresponding collections of provider services as their concrete dynamic service systems.

A *context service* is a collection of *named entries* such that each name is uniquely associated with a constant value (*value entry*), evaluated value (*evaluated entry*), or exerted value (*service entry*). An evaluated or exerted entry in its context-service scope is a functional composition that depends on other entries of that scope. Given the set *ES* of all entries, the set of context services *CSS* is the powerset of entries $\mathcal{P}(ES)$. Context services with value entries only are called *data model services* and denoted by *DMS*. Context services that contain service entries are called *context model services* and denoted by *CMS*. Therefore, the set of context services *CSS* is the union

$$CSS = DMS \cup CMS. \quad (1)$$

The *context realization* r_c is the following mapping:

$$r_c: CSS \times DMS \longrightarrow \{ true, false \} \quad (2)$$

that defines the meaning of context services. If the realization $r_c(incss, outdms)$ of input context model service *incss* into output data model service *outdms* equals to *true*, then

it is said that context model *incss* is *valid* for the data model *outdms*, otherwise is *invalid*.

The elementary service realization r_e is the mapping

$$r_e : MISS \longrightarrow PSS \quad (3)$$

where *MISS* is a set of microservices and *PSS* is a set of provider services.

The set of all component services for macroservice *mas* is given by the function *cs*:

$$cs : MASS \longrightarrow \mathcal{P}(MASS) \quad (4a)$$

and the set of all component microservices for macroservice *mas* is given by the function *cms*

$$cms : MASS \longrightarrow \mathcal{P}(MISS) \quad (4b)$$

The *federated binding* *b* is the mapping defined as follows:

$$b : MASS \longrightarrow \mathcal{P}(PSS) \quad (5)$$

such that for each macroservice $mas \in MASS$ and each $mis \in cms(mas)$ (4b), $r_e(mis) \in b(mas)$, where *MASS* is the set of macroservices and $\mathcal{P}(PSS)$ is the powerset of provider services. The set of provider services $b(mas)$ is called a *service federation* of macroservice *mas*.

For each macroservice $mas \in MASS$ multiple service federations F_i , $i = 1, 2, \dots, n$ are actualized by the *federated realization* r_f as follows:

$$r_f : MASS \longrightarrow \{F_i\} \quad (6)$$

such that for each federations F_i exists the binding $b(mcs) = F_i$. Multiple federations for a macroservice are in existence to provide system reliability in the presence of provider service replication.

The macroservice such that any named subset of its component services is used as a service is called a *multifidelity service*. A *service projection* p_s of the *multifidelity service* *mfs* is the mapping:

$$p_s : FI \times MFSS \longrightarrow \mathcal{P}(cs(mfs)) \quad (7)$$

such that $p_s(fi, mfs) \in \mathcal{P}(cs(mfs))$ (4a), where *FI* is the set of fidelities, $fi \in FI$, *MFSS* the set of multifidelity services, $mfs \in MFSS$, and $cs(mfs) \subseteq MASS$ is the set of all component services of *mfs*. In particular, fidelities may refer to singletons of $cs(mfs)$.

The set of *all macroservice MASS* is a set defined as follows: (8)

- i. if $s \in MISS$ then $\{s\} \in MASS$
- ii. if $s \in CSS$ then $\{s\} \in MASS$
- iii. if $s \in MFSS$ then $\{s\} \in MASS$
- iv. $s \in \mathcal{P}(MASS)$ then $s \in MASS$

Each macroservice is a set associated with a hierarchically organized *component services* as defined in i-iv. In other words, a *macroservice* is a *collection of services* hierarchically composed of context, micro, multifidelity, and other macroservices. Multifidelity services are associated with *service projections* (7) that can be invoked at runtime to create another instance of the metasystem. Each macroservice with federated realization r_f (6) can be *bound* to at least one federation of available provider services. When service fidelities are updated then related service federations adapt in size and quality as needed.

A *morphing projection* p_m is the mapping

$$p_m : MFI \longrightarrow \mathcal{P}(FI) \quad (9)$$

assigning for each metafidelity a corresponding set of service fidelities, where MFI is the set of all metafidelities and $\mathcal{P}(FI)$ is the powerset of all service fidelities such that for each metafidelity mfi , $p_m(mfi) \subseteq FI$, for $mfi \in MFI$.

Projection functions p_s and p_m define service and metafidelities service that allow for adaptivity of system-of-systems when updates of fidelities are controlled by fidelity management at runtime as illustrated in **Figure 1**.

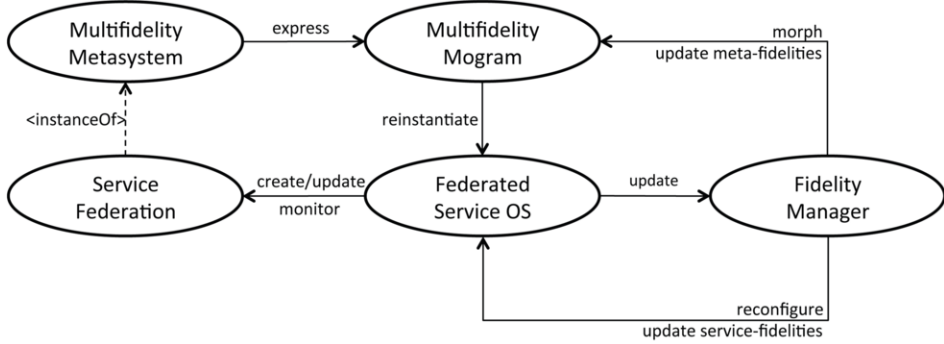


Figure 1. Morphing and reconfiguring mograms by a federated service operating system (OS)

Adaptive federated SO systems with varying metafidelities at runtime are called *federated amorphous systems*. This type of systems exhibits three types of adaptivities called *system-of-system* (metasystem), *system*, and *service* agilities. Metasystem agility refers to updating metafidelities (system reinstantiation - morphing), system agility refers to updating service fidelities (system reconfiguration), and service agility refers to selecting relevant fidelities of individual provider services.

1.2. Federated Amorphous Systems

The federated amorphous system (FAS) is a triplet (service system, multifidelity system, system-of-systems) of triplets:

$$FAS = \langle \langle CSS, MISS, PSS \rangle, \langle MFSS, FI, p_s \rangle, \langle MFI, p_m, \langle r_c, r_e, r_f \rangle \rangle \rangle \quad (10)$$

where

- $CSS, MISS, PSS, MFSS$ are sets of context, micro, provider, and multifidelity services respectively;
- Macroservices are compound services with hierarchically organized component services in $MASS$ (8);
- $p_s: FI \times MFSS \rightarrow \mathcal{P}(MASS)$ is a *service projection* (7) such that $p_s(fi, mfs)$ is the subset of component services of mfs , where fidelity $fi \in FI$ and multifidelity service $mfs \in MASS$;
- $p_m: MFI \rightarrow \mathcal{P}(FI)$ is the *morphing projection* (9) such that for each metafidelity mfi , $p_m(mfi) \subseteq FI$, for $mfi \in MFI$;
- $r_c: CSS \times DMS \rightarrow \{ true, false \}$ is the *context realization* (2) that defines the meaning of context models;

- f) $r_e: MISS \rightarrow PSS$ is the *elementary service realization* (3) such that for each $s \in MISS$, $r_e(s) \in PSS$;
- g) $r_f: MASS \rightarrow P(PSS)$ is the *federated realization* (6) such that for each $s \in MASS$, $r_f(s) \in P(PSS)$;
- h) *service reification* in *FAS* is the mapping $ri: MASS \rightarrow PSS$ such that a macroservice $mas \in MASS$ is converted into a provider service $ri(mas) \in PSS$;
- i) *service provisioning* in *FAS* is the mapping $pr: MASS \rightarrow P(PSS)$ such that $pr(mas) = r_f(mas)$ and $pr(mas) \subseteq PSS$ for each macroservice $mas \in MASS$; and
- j) *evaluation of microservice* $mas \in MASS$, is the mapping

$$e: MASS \times CMS \times P(MFI) \times P(FI) \rightarrow DMS \quad (11)$$

such that for macroservice mas and input model service $incm \in CMS$, $MFI_{mas} \subseteq MFI$, and $FI_{mas} \subseteq FI$, exists federation $F_{mas} \subseteq P(PSS)$ and the corresponding output model service $outdm = e(mas, incm, MFI_{mas}, FI_{mas})$ provided $r_f(mas) = F_{mas}$ and $r_c(incm, outdm) = true$. Service evaluations are consistent with the morphing/service projections, service reification, and provisioning functions. The feedback to parent services how to morph/update fidelities is driven by observed actions of provider services. In particular, evaluation of context service $cs \in CSS$ is the mapping

$$e: CSS \times CMS \times P(MFI) \times P(FI) \rightarrow DMS \quad (12)$$

such that for each context model $cm \in CSS$ the corresponding output model service $outdm = e(cm, incm, MFI_{mas}, FI_{mas})$, provided $incm$ is the context scope for each entry in cm to be evaluated as the functional composition with dependent entries in both cm and $incm$.

The federated service system reduced to a *client-server system* (C/S) is defined as follows:

$$FAS = \langle\langle DMS, \{ client \}, \{ server \} \rangle, \langle r_c, r_e, r_f \rangle \rangle \quad (13)$$

where the set of microservices *MISS* is reduced to one *client* and provider services *PSS* is reduced to one provider *server* with $r_e(client) = server$, $r_f(client) = \{ server \}$, and r_c is just the server mapping from data input to data output. The end users in C/S do not have any frontend control regarding how the server would collaborate with other servers. No frontend macroservices, only single microservice (client) to the static backed seen as the black box by the end users.

When all fidelities in *FAS* do not change at runtime, the system is called the *static* multifidelity *FAS*, otherwise is called the *amorphous FAS*. System realization and configuration functions: r_c , r_e , r_f , p_s , p_m , ri , and pr define the essential dynamic configuration support for managing amorphousness of the complex adaptive *FAS*.

Note that a hierarchically organized macroservices are evaluated with context services as evaluation arguments. Here, service evaluation arguments – data or context models – are services as well. On the one hand, macroservices can be defined as hierarchically composed services (8). On the other hand, values of context model entries can be macroservices and then used as arguments of functional compositions defined by other entries within the same context service (1). The former compositions are called exertions; the latter are called context models. Defining service data and operations as uniform system of services is referred to the “true service-oriented

architecture” in contrast to other so called “service-oriented” architectures lacking this type of service uniformity.

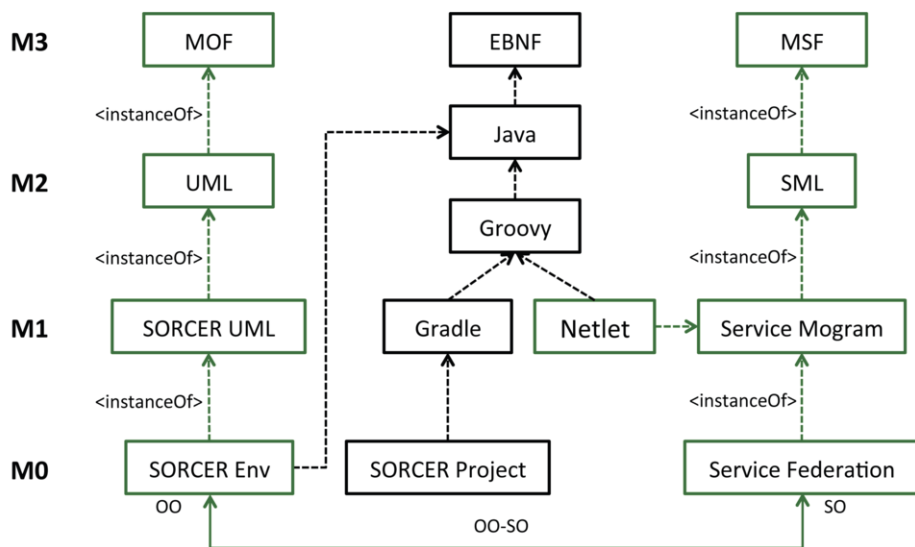
2. Service-oriented Computing Environment (SORCER)

2.1. The SORCER Four-layer Metamodel Hierarchy

The SORCER meta-metamodeling layer, called SORCER Meta Service Facility (MSF) is the foundation of the service-oriented metamodeling hierarchy by the analogy to the object-oriented the metamodeling hierarchy of OMG Meta Object Facility (MOF) [3]. The primary responsibility of this layer is to define the language for specifying a metamodel. The layer is often referred to as M3, and FAS is an example of a meta-metamodel. A mathematical meta-metamodel is typically more compact than a metamodel - SML (Service Modeling Language). An SML is an instance of FAS, meaning that every element of SML is an instance of an element in FAS. The primary responsibility of the SML layer is to define a language for specifying service models in the form of service mograms. The metamodel layer is often referred to as M2; the UML metamodel is an instance of the MOF and the SML metamodel is an instance of the MSF as illustrated in **Figure 2**.

A model is an instance of a metamodel. The primary responsibility of the model layer is to define languages that describe semantic domains to allow users to model a wide variety of different problem domains in particular to express various types of SO processes. The things that are being modeled reside outside the metamodel hierarchy. This layer is often referred to as M1. A user object model is an instance of the UML metamodel and a service model is an instance of SML metamodel.

The metamodel hierarchy bottoms out at M0, which contains the run-time instances of model elements defined in a model. The SORCER UML model of SML is implemented in Java (SORCER API) and is used to implement all service-oriented concepts of MSF.



Languages can be specified by metamodeling with a great flexibility, as shown in **Figure 2** for UML and SML. They can be also specified by grammars, for example the Java language in EBNF. Interpreted SML mograms, restricted to the Groovy language syntax are called netlets so scripting is available in SORCER due to the compatibility of Groovy and Java since SML operators are implemented in Java API of the SORCER environment. The relationship OO-SO in **Figure 2** shows how object orientation in UML (backend OO provider services) meets service orientation) in SML (frontend SO macroservices).

2.2. The SORCER Platform

Computing requires a platform (runtime system) to operate. Computing platforms that allow programs to run require a processor, operating system, and programming environment with supporting tools to create and run programs. SORCER is the federated SO environment that implements FAS as defined in (10). The SORCER programming environment is based on the *Front-end as a Service* (FaaS) *architecture* with its unique *service-oriented operating system* and associated *service processor* as the collection of provider services (3). Technically, the service processor comprises Java local/remote objects implementing service types incorporated by microservices. Remote objects are deployed with dynamic small-footprint dynamic service containers called *service providers*.

Macroservices are specified in DSL called the Service Modeling Language (SML) that consists of two parts: Context Modeling Language (CML) and Exertion-Oriented Language (EOL). Macroservices expressed in imperative EOL - service programs - are called *exertions*. Macroservices expressed in declarative CML - collections of interrelated functional compositions - are called *context models*. In other words exertions are service collaborations with an explicit flow of control while models with explicit functional dependencies (no explicit control flow). A *macroservice* comprising of both context models and exertions is called a *service mogram* [8][9].

Mogram-based FaaS is a computing architecture that provides the end user the direct access to a dynamic collaboration of applications, servers, storage, databases and other resources as a backend *federation of services* (6). This architecture was introduced in the FIPER project [10] with exertion-oriented programming for creating frontend imperative services (exertions). The SORCER environment extends that approach by unifying exertion-oriented programming with context modeling (declarative programming) [11]. The SORCER Operating System (SOS) implements the FAS functions r_e , r_f , ri , pr , and e as defined in (10, 11, 12). Runtime federations defined by mograms that are created and managed by SOS constitute its dynamic service processor [12].

SORCER dynamically federates backend services and provides common features for network integration and location services provided by SOS [2]. An alternative to FaaS, is a common Backend as a Service (BaaS) approach that uses application programming interfaces (APIs) and software developer's kits to connect static user agents to static backend network resources. FaaS is the departure from typical application development, which requires software developers to incorporate the APIs of each backend service individually into larger backend services available to the end user. FaaS is focused on service-oriented complex adaptive models created by the end users in SML as illustrated in **Figure 3**.

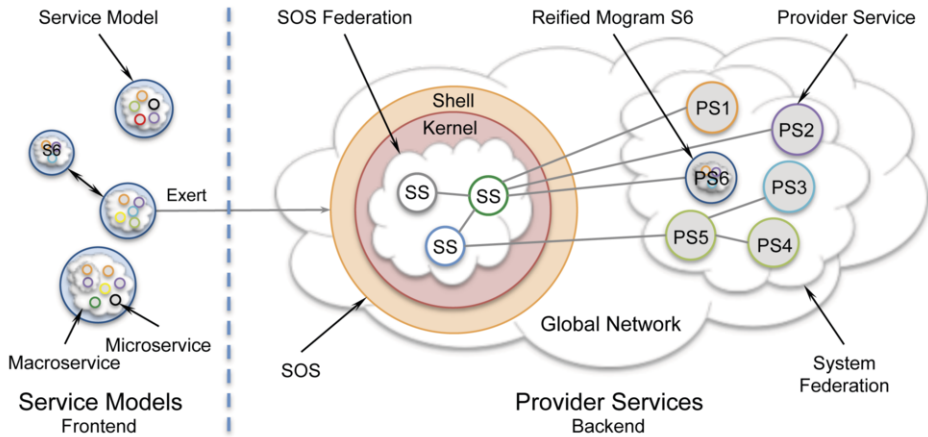


Figure 3. An exerted service model runs the system federation managed dynamically by SOS as defined by the end user. SOS binds microservices of the exerted macroservice dynamically to corresponding provider services. SOS also reifies the microservices, for example S6 as PS6, while missing services are provisioned on-demand as required if needed.

3. N³ diagramming and Service-oriented Mogramming

Top-down and bottom-up problem solving describes two different methods of thinking: working at the top is considered strategic and declarative, while working at the bottom is tactical and imperative. How a given situation is actually perceived and processed will vary with the person, experience, and runtime environment chosen. However, the approach is to do whatever is best for managing complexity of the solution by a combination of both declarative and imperative thinking.

In declarative programming a process is expressed by a functional composition while in imperative programming is expressed by an algorithm. An algorithm is a procedure for solving a problem in the form of a self-contained step-by-step set of services (operations) to be performed with explicit control flow defined. The emphasis on explicit control flow distinguishes an imperative programming language from a declarative programming language. Service mogramming can start with either a declarative context model or an imperative exertion then can expend into the composite mogram by adding component services of either type as it fits the composition better.

The N² diagram [14] design structure represents the functional or physical interfaces between system elements depicted as diagonal services with connectors showing data flow between services. An expanded N² diagram with diagonal services that can be hierarchically organized with component diagrams representing service mograms along with multifidelity services and flow of control of exertions is called an N³ (N-cubed) diagram [5]. An example of an N³ diagram is depicted in **Figure 4**. It represents the following functional composition of services:

$$[g: e_{1,2}; e_{6,3}]e_{6,*}(e_{1,2}(m_x), m_1(e_{1,1}, m_{y,2}), m_{3,1}(e_{1,1}, e_{z,2}, e_t)) \quad (9)$$

where $[g: e_{1,2}; e_{6,3}]$ denotes a guard for e_6 defining a loop under condition g : if g is true then $e_{1,2}$ else $e_{6,3}$. A current fidelity $e_{6,*}$ of multifidelity exertion e_6 is determined by

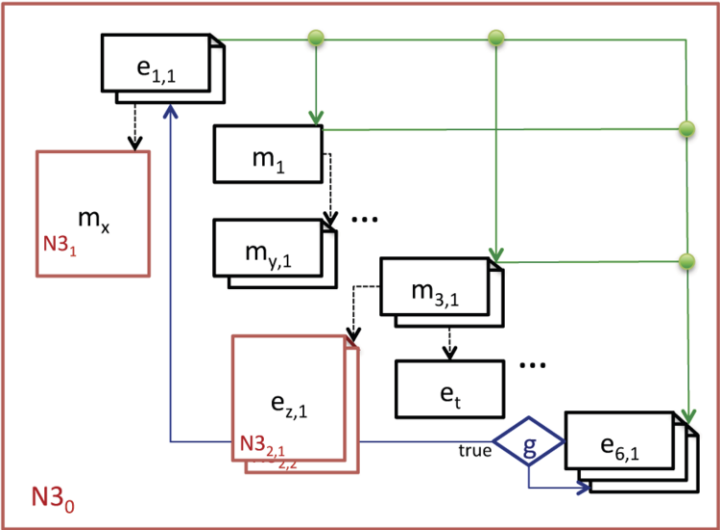


Figure 4. N^3 diagram for the composition: $[g: e_{1,2}; e_{6,3}]e_{6,*}(e_{1,2}(m_x), m_1(e_{1,1}, m_{y,2}), m_{3,1}(e_{1,1}, e_{z,2}, e_t))$

this self-aware service at runtime. The third dimension here is represented by multifidelities of service nodes: exertion e_1 , model m_3 , and exertion e_6 with fidelities $e_{1,1}$ and $e_{1,2}$ for exertion e_1 ; $m_{3,1}$ and $m_{3,2}$ for model m_3 ; and $e_{6,1}$, $e_{6,2}$, and $e_{6,3}$ for exertion e_6 . Additionally, each service node can be hierarchically nested with its own N^3 diagrams, for example $m_{3,1}$ depends on N^3 diagram $N3_2$, exertion $e_{1,2}$ on $N3_1$, context model m_1 on multifidelity service m_y , and service model $m_{3,2}$ on exertion e_t . The matrix crossing points (small circles) represent connectors that match outputs to relevant inputs between heterogeneous and autonomous services.

In SORCER a SO process represented by an N^3 diagram can be defined declaratively with CML as a *context model* or procedurally as an *exertion* with EOL, or with both languages at the same time as a *mogram*. Within EOL control flow exertions (branching and looping exertions) are statements whose execution results in choice being made as to which of two or more execution paths should be followed. Top-level mograms (as the root mogram in **Figure 4**) representing N^3 diagrams can be implemented either way, with declarative (CML) or imperative (EOL) mogramming semantics.

The first rule of distributed systems is don't distribute your system until you have an observable reason to do so. The concept of SORCER service signature used in models and exertions supports this rule. In a service signature first you specify your class type that implements your service type rather than the interface type of service. When you have an observable reason for system distribution then you can change the service implementation type to the interface type in signatures to have your local service modules to be remote services. Federated service-orientation is all about bringing object-orientation to the network the way local and remote objects (provider services) are alike in service collaborations specified by mograms and created by the end users. A mogram created at the frontend executes a dynamic federation at the backend.

4. Conclusions

The presented approach to ATE is based on federated service systems with multifidelity management. The SORCER net-centric reference architecture of such systems is about bringing object-orientation to the network with exchangeable local and remote provider services that are used uniformly in creating scalable service collaborations at runtime. That is consistent with the first rule of distributed systems: don't distribute your system until you have an observable reason to do so. The SORCER environment allows you to create a SO monolith (for local service federations only) and instantly distribute it step by step as needed later.

The SML modeling language of amorphous ATE systems is service provider agnostic — it is not concerned with the physical computing resources but rather with the expression of amorphous models of service federations as metamodeling abstractions (mograms) with the goal of understanding both high-fidelity physics-based modeling and emergent behavior of complex adaptive systems. SORCER is implemented in Java and provides Java API to create objects that do not correspond directly to the presented modeling conceptualizations reflected by declarative (CML) and imperative (EOL) DSLs. Java API is not the SORCER modeling language, it is used in combination with CML/EOL to implement provider services (providers) along with other languages compiled into executable codes wrapped by the providers. A mogram, created by the end users, represents a system of system instances (dynamic federations of provider services at runtime) created and morphed by the SORCER operating system.

The evolving SORCER core platform (the GitHub open source project [13]) provides front-end mogramming languages [9], Java API, and the operating system [2] that implements required realizations of FAS (10). The presented federated approach has been verified and validated in research projects at the Multidisciplinary Science and Technology Center, AFRL/WPAFB [15][16][17][18].

Acknowledgement

This work was partially supported by Air Force Research Lab, Aerospace Systems Directorate, Multidisciplinary Science and Technology Center, the contract number FA8650-10-D-3037, Service-Oriented Optimization Environment for Distributed High Fidelity Engineering Design Optimization; by Polish-Japanese Academy of Information Technology; and by the Natural Science Foundation of China (Project No. 51175033).

References

- [1] Kolonay, R. M., (2014). A physics-based distributed collaborative design process for military aerospace vehicle development and technology assessment, *International Journal on Agile Systems and Management*, Vol. 7, Nos. ¾
- [2] Sobolewski, M., (2015). Technology Foundations. In: J. Stjepandić et al. (eds.) *Concurrent Engineering in the 21st Century*, ISBN 978-3-319-13775-9, Springer International Publishing Switzerland, pp 67-99
- [3] OMG Meta Object Facility (MOF) Core Specification, (2015). Object Management Group. Retrieved April 27, 2016, <http://www.omg.org/spec/MOF/2.5/PDF>

- [4] Sobolewski, M., (2014). Service oriented computing platform: an architectural case study. In: Ramanathan R, Raja K (eds) *Handbook of research on architectural trends in service-driven computing*, IGI Global, Hershey, pp 220-255
- [5] Sobolewski, M., Kolonay, R. (2015). Service-oriented Life Cycles for Developing Transdisciplinary Engineering Systems, In: Curran, R., et al. (eds) *Transdisciplinary Lifecycle Analysis of Systems*, Proceedings of the 22nd ISPE Inc. International Conference on Concurrent Engineering, July 20-23, 2015, ISBN 978-1-61499-543-2, IOS Press, pp 541-551
- [6] Moulay Aziz-Alaoui & Cyrille Bertelle (eds), (2006) *Emergent Properties in Natural and Artificial Dynamical Systems (Understanding Complex Systems)*, ISBN-13: 978-3540348221, Springer
- [7] Delligatti, L., (2013) *SysML Distilled: A Brief Guide to the Systems Modeling Language*, ISBN-13: 978-0321927866, Addison-Wesley Professional
- [8] Kleppe A., 2009. *Software Language Engineering*, Pearson Education, ISBN: 978-0-321- 55345-4
- [9] Sobolewski, M. and Kolonay, R., 2012. Unified Mogramming with Var-Oriented Modeling and Exertion-Oriented Programming Languages, *Int. J. Communications, Network and System Sciences*, 2012, 5, 9. Published online <http://www.scirp.org/journal/PaperInformation.aspx?paperID=22393>
- [10] Sobolewski, M., (2002) Federated P2P Services in CE Environments, *Advances in Concurrent Engineering*, A.A. Balkema Publishers, 2002, ISBN 90 5809 502 9, pp. 13-22
- [11] Sobolewski, M., (2014). Unifying Front-end and Back-end Federated Services for Integrated Product Development, In: J. Cha et al. (eds) *Moving Integrated Product Development to Service Clods in Golbal Economy*, IOS Press. Pp 3-16. Retrieved May 25, 2015, <http://ebooks.iospress.nl/publication/37838>
- [12] Sobolewski, M., 2012, Object-Oriented Service Clouds for Transdisciplinary Computing, in I. Ivanov et al. (eds.), *Cloud Computing and Services Science*, DOI 10.1007/978-1-4614-2326-3_1, Springer Science + Business Media New York
- [13] SORCER Project. Retrieved April 28, 2016, <http://sorcersoft.org/project/site/>
- [14] Browning, T.R. (2001) Applying the Design Structure Matrix to System Decomposition and Integration Problems: A Review and New Directions, *IEEE Transactions on Engineering Management*, Vol. 48, No. 3, pp 292-306
- [15] Burton, S.A., Alyanak, E.J., and Kolonay, R.M. (2012). Efficient Supersonic Air Vehicle Analysis and Optimization Implementation using SORCER, 12th AIAA Aviation Technology, Integration, and Operations (ATIO) Conference and 14th AIAA/ISSM AIAA 2012-5520
- [16] Sobolewski, M., Burton, S., & Kolonay, R. (2013). Parametric Mogramming with Var-oriented Modeling and Exertion-Oriented Programming Languages. Proceedings of the 20th ISPE International Conference on Concurrent Engineering, C. Bil et al. (Eds.), ISBN: 978-1-61499-301-8 (print), 978-1-61499-302-5 (online), IOS Press, 2013 pp. 381-390. Retrieved April 28, 2016, <http://ebooks.iospress.nl/publication/34826>
- [17] Kolonay, R. M. and Sobolewski M., 2011. Service ORiented Computing EnviRonment (SORCER) for Large Scale, Distributed, Dynamic Fidelity Aeroelastic Analysis & Optimization, *International Forum on Aeroelasticity and Structural Dynamics*, IFASD2011, 26-30 June, Paris, France
- [18] Kolonay, R. M., Thompson, E. D., Camberos, J. A. and Eastep, F., 2007. Active Control of Transpiration Boundary Conditions for Drag Minimization with an Euler CFD Solver, *AIAA-2007-1891, 48th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, Honolulu, Hawaii